

Basic Linux Commands			Setup & Staging		
Command	Sample usage	What it does	Command	Sample usage	Description
pwd	pwd	Print the current working directory path	git init	git init	Turns the current folder into a new Git repo Flags: --bare
ls	ls -la	List files/folders (all, long format)	git clone	git clone <url>	Downloads a full copy of a remote repo + history Flags: --branch, --depth 1
cd	cd projects/game	Change into a different directory			Shows staged, changed, and untracked files Flags: -s (short)
mkdir	mkdir assets	Create a new directory (-p for nested paths)	git status	git status	Stages changes for the next commit Flags: -A (all), -p (interactive)
rm	rm -rf old_build/	Remove files (-r folders, -f force)	git add	git add index.php style.css	Saves staged snapshot to history Flags: -rm, -a, --amend
cp	cp index.html bak.html	Copy a file or directory (-r for folders)	git commit	git commit -m "Fix bug"	Reverts a file to its last committed state Flags: --staged (unstage)
mv	mv game.js src/game.js	Move or rename a file/directory	git restore	git restore levell.js	Removes/renames a tracked file and stages it Flags: --cached (keep on disk)
cat	cat config.php	Print a file's contents to the screen	Undoing & Staging		
touch	touch .gitignore	Create an empty file, or update its timestamp	Command	Sample usage	Description
grep	grep -rn "TODO" src/	Search text for a pattern (-r recursive)	git reset	git reset --soft HEAD~1	Moves HEAD back; optionally un-stages/discards Flags: --soft, --mixed, --hard
find	find . -name "*.js"	Search for files by name or pattern	git revert	git revert a1b2c3d	Makes a new commit that undoes an old one (safe for shared history)
chmod	chmod +x deploy.sh	Change a file's permissions	git stash	git stash / stash pop	Shelves uncommitted changes to reuse later Flags: -u (incl. untracked)
ps	ps aux	List currently running processes	History & Branching		
kill	kill -9 1234	Terminate a process by its PID	Command	Sample usage	Description
tar	tar -czvf out.tar.gz dir/	Compress/extract archive files (-x extract)	git log	git log --oneline --graph	Lists past commits, newest first
df	df -h	Show available disk space, human-readable			
du	du -sh folder/	Show how much space a folder is using			
wc	wc -l file.txt	Count lines/words/characters in a file			
history	history	List recently run terminal commands			
sudo	sudo apt install git	Run a command with admin/root privileges			
ssh	ssh user@server.com	Log in to a remote machine securely			
curl	curl -O https://.../file.zip	Transfer data / download from a URL			
man	man git-commit	Open the manual page for a command			

More Linux Essentials		
Command	Sample usage	What it does
alias	alias ll='ls -la'	Create a shortcut name for a longer command
export	export PATH=\$PATH:~/bin	Set an environment variable for the session
which	which node	Show the full path to a command's executable
whoami	whoami	Print the currently logged-in username
uname	uname -a	Show system, kernel, and architecture info
top / htop	top	Show live, real-time system resource usage
ln	ln -s target.js link.js	Create a symbolic (shortcut) link to a file
wget	wget https://.../file.zip	Download a file from a URL to disk
unzip	unzip archive.zip	Extract the contents of a .zip archive
apt	sudo apt update && sudo apt upgrade	Refresh and update installed packages (Debian/Ubuntu)
echo	echo "hello" > note.txt	Print text to the screen or into a file
nano / vim	nano config.php	Open a file in a simple terminal text editor

Git's Three-Tree Architecture

Every tracked file moves through three areas — this is exactly what `add`, `commit`, and `restore/reset` move files between.

1. Working Directory

The actual files you see and edit on disk right now. Untracked/modified changes live here first.

2. Staging Area (Index)

A holding area listing exactly what goes into the *next* commit. Filled by `git add`.

3. Repository (.git / HEAD)

Permanent, committed snapshots — Git's saved project history. Filled by `git commit`.

— `git add` →

← `git restore / checkout` —

In short: **edit** in the working directory → **stage** the snapshot you want → **commit** it permanently to history. Moving backward (`restore/checkout/reset`) copies files the other direction.

File Permissions (chmod)		Terminal Shortcuts		Handy Git Workflows	
Mode	Meaning	Keys	Effect	Goal	Command
755 / <code>rwxr-xr-x</code>	Owner: read/write/execute — others: read/execute. Typical for scripts & folders	<code>Ctrl + C</code>	Cancel/kill the running command	Undo last commit, keep changes	<code>git reset --soft HEAD~1</code>
644 / <code>rw-r--r--</code>	Owner: read/write — others: read only. Typical for regular files	<code>Ctrl + D</code>	Exit the shell / end input (EOF)	Discard all local changes	<code>git restore .</code>
		<code>Ctrl + R</code>	Search backward through command history	Rename current branch	<code>git branch -m new-name</code>
700 / <code>rwX-----</code>	Owner-only access — private files/keys	<code>Ctrl + L</code>	Clear the terminal screen	Delete a merged branch	<code>git branch -d old-branch</code>
		<code>Ctrl + Z</code>	Suspend the running process (resume with <code>fg</code>)	See who last edited each line	<code>git blame file.js</code>
<code>chmod u+x file</code>	Symbolic form: add execute permission for the owner	<code>Ctrl + U</code>	Clear the line before the cursor	Compare two branches	<code>git diff main..feature</code>
		<code>Ctrl + A / E</code>	Jump cursor to start / end of the line	Remove untracked files/folders	<code>git clean -fd</code>
I/O Redirection & Pipes		Tab	Auto-complete file/command names	Fix the last commit message	<code>git commit --amend -m "msg"</code>
		<code>↑ / ↓</code>	Cycle through previous commands	Sync a fork with upstream	<code>git fetch upstream; git merge upstream/main</code>
Operator	Meaning	<code>!!</code>	Repeat the last command run		
<code>cmd > out.txt</code>	Send output to a file, overwriting it				
<code>cmd >> out.txt</code>	Append output to the end of a file				
<code>mysql db < dump.sql</code>	Feed a file in as input				
<code>cat log grep err</code>	Pipe one command's output into another				
<code>cmd1 && cmd2</code>	Run cmd2 only if cmd1 succeeds				
<code>npm run dev &</code>	Run a command in the background				
.gitignore Quick Guide					
Common patterns to exclude from version control					
node_modules/ — installed JS packages					
vendor/ — installed PHP/Composer packages					
*.log — log files of any name					
.env — local secrets/config, never commit					
.DS_Store — macOS Finder junk files					

MySQL Quick Reference		Deploying Your Web Game	
Command	What it does	Command	What it does
mysql -u root -p	Open an interactive MySQL shell	ssh user@server.com	Connect to your web/game server
SHOW DATABASES;	List all databases on the server	scp build.zip user@srv:/path	Copy a single file to a remote server
USE game_db;	Switch to a specific database	rsync -avz ./ user@srv:/var/www/	Sync a whole folder, only changed files
SHOW TABLES;	List tables in the current database	composer install	Install PHP dependencies from composer.json
SELECT * FROM scores LIMIT 10;	Query the first 10 rows of a table	npm install	Install JavaScript dependencies from package.json
mysqldump -u root -p db > b.sql	Export/back up a database to a file	npm run build	Bundle/minify JS & CSS for production
mysql -u root -p db < b.sql	Import a SQL file into a database	git pull origin main	Pull latest code straight onto the server

PHP & Composer Quick Reference		Cron & Scheduling	
Command	What it does	Command	What it does
php -v	Check the installed PHP version	crontab -e	Edit your scheduled tasks (cron jobs)
php -S localhost:8000	Start PHP's built-in dev server for local testing	crontab -l	List currently scheduled cron jobs
composer init	Create a new composer.json for the project	0 2 * * * backup.sh	Example: run backup.sh every day at 2:00 AM
composer require monolog/monolog	Add a new PHP package as a dependency	Vim Basics (when nano isn't enough)	
composer update	Update dependencies to their latest allowed versions	Keys	Effect
composer dump-autoload	Rebuild the class autoloader after adding files	i	Enter insert (typing) mode
		Esc	Return to normal (command) mode
		:wq	Save changes and quit
		:q!	Quit without saving changes
		dd / yy / p	Delete / copy / paste the current line